FYP-1 · MEETING 2 · 24-OCT-26 · SAMPLE FOR ALL GROUPS

Sample backend architecture: CivicFix

CivicFix is an invented project, used only to show the standard expected on 24-Oct. Citizens report civic problems (potholes, garbage, water leaks, broken streetlights) with a photo, a voice note and their location, by app or WhatsApp. AI classifies each report, removes duplicates and sets a priority. The system routes it to the right municipal department, assigns the nearest field worker and escalates it if the deadline is missed.

Every member presents, and every

member is marked

4 × 2.5

minutes: each member gets about 2.5 minutes for their part, around 2 minutes presenting and 30 seconds answering one question from the class.

10

minutes per group, hard stop. Rehearse with a timer. Talking past your time eats your teammate's marks.

1 by 1

Individual marks. Each member is scored on their own part. A member who doesn't present gets no presentation marks for that meeting.

This is a sample, not a template. Copy the standard and the way the parts are split, not the components. Your architecture must follow from your own project's features.

The architecture: event-driven microservices

Your presentation must **name your architecture style** in the first minute and **say why you chose it** over at least one alternative. Here is how CivicFix does it.

Event-driven microservices behind an API gateway

Small services, each owning one job, sit behind one gateway. Slow work (AI, notifications, deadline checks) is handed off as **events** through a message queue instead of being done while the user waits.

Why we chose it

- **AI takes 2–4 seconds per report.** A citizen should get a reply in under a second, so the AI must run in the background. Events make that natural.
- **Traffic is spiky.** Monsoon rain brings 10× the complaints in a day. We can add more AI workers without touching anything else.
- **Mixed languages.** The AI is in Python and the API is in Node.js, so separate services let each use its best tools.

Monolith (layered / MVC)

Simplest to build and deploy, but the AI would block requests and we couldn't scale it separately. *A fine choice if your project has no slow or heavy processing.*

Serverless / BaaS (e.g. Firebase)

Fast to start, but long AI jobs, PostGIS queries and a message queue don't fit well. *A fine choice for mostly CRUD apps.*

Microservices without events

Services calling each other directly would chain their failures: if AI is down, complaint submission fails too.

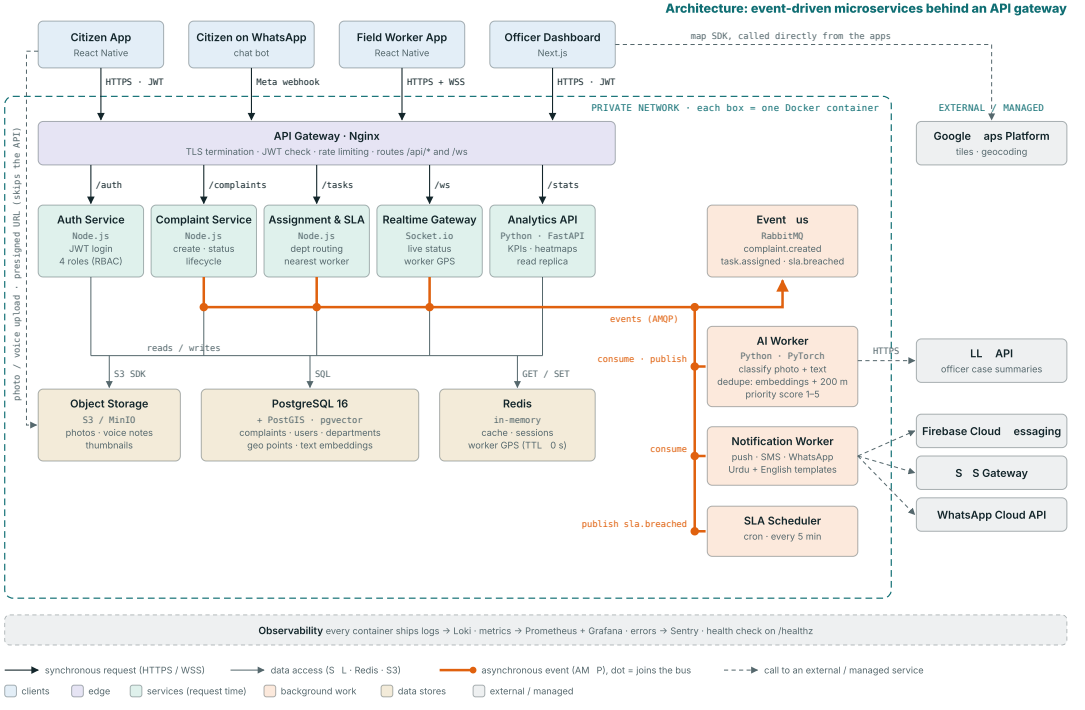
Common styles to choose from: **layered monolith (MVC)** · **modular monolith** · **client + BaaS / serverless** · **microservices** · **event-driven** · **pipeline (for AI / data processing)** · **agent-based (LLM + tools)**. Many projects combine two. Say which, and why.

The diagram

The full-size diagram is on the next page. Rotate your phone and zoom in.

CivicFix · event-driven microservices behind an API gateway

zoom in · every arrow is labelled



Follow one complaint through the system

Every good architecture talk follows one real request end to end. Steps marked `async` happen after the citizen already has a reply.

1. The app gets a **presigned upload URL** and sends the photo and voice note **straight to object storage**. Large files never pass through the API.
2. The app posts the complaint (text, GPS, file keys) through the **API Gateway**, which checks the JWT and rate limit and routes it to the Complaint Service.
3. The Complaint Service saves it in **PostgreSQL**, publishes `complaint.created` and returns a tracking number in about 150 ms.
4. The **AI Worker** classifies the photo and text and checks for duplicates: same category, within 200 m, and similar text. `async`
5. It publishes `complaint.classified` with a category and a priority from 1 to 5. A duplicate is merged into the existing case. `async`

6 **Assignment & SLA** picks the department and the nearest free worker (live GPS in Redis, distance in PostGIS). `async`

7 The **Notification Worker** alerts the worker and the citizen. The officer dashboard updates live over WebSocket. `async`

8 The worker streams GPS every 15 s, uploads an "after" photo and marks the job `RESOLVED`.

9 Every 5 minutes the **SLA Scheduler** finds overdue cases and escalates them to a supervisor. `async`

How to split the parts among yourselves

Split by layer, not by slide. Each member teaches one concept from their layer, points at it on the diagram, then takes one question. Match parts to each member's expertise.

Member 1	Member 2	Member 3	Member 4
0:00–2:30	2:30–5:00	5:00–7:30	7:30–10:00

MEMBER 1 · 0:00–2:30

Architecture style, clients and the request path

- Open with the **name of the architecture** and why it was chosen over a monolith (see above).
- Four clients, one front door: the gateway checks TLS, the JWT and the rate limit once for everyone.
- Teach the concept: **presigned URLs**, so a 5 MB photo goes straight to storage and skips the API.

Point at: client arrows, the dashed upload path, gateway routes.

Likely question: Why not just use one big backend?
Answer: AI would block requests, and we couldn't scale it separately at peak time.

MEMBER 2 · 2:30–5:00

Services and where data lives

- Each service owns one job. For example, Auth issues tokens but never touches complaints.
- Teach the concept: **choosing a data store by access pattern**. PostgreSQL for records that must be correct, PostGIS for "nearest", pgvector for "similar", Redis for data that expires.
- The Analytics API reads from a replica, so dashboards never slow down new complaints.

Point at: the five services, the grey data lines, the three stores.

Likely question: Why PostgreSQL and not MongoDB?
Answer: Complaints, workers and departments are related, and "nearest worker" is one indexed PostGIS query.

MEMBER 3 · 5:00–7:30

Async events and the AI pipeline

- Teach the concept: **synchronous vs asynchronous**. AI takes 2–4 s, so it runs off the request path.
- Publish/subscribe: the Complaint Service doesn't know who listens, so new consumers plug in freely.
- Walk steps 4–7: classify → dedupe → prioritise → assign → notify.

Point at: the orange path from the services to the event bus and the workers.

Likely question: What if the queue goes down?
Answer: Complaints are still saved. Events wait in an outbox table and are sent once the queue is back.

MEMBER 4 · 7:30–10:00

Security, scaling and running it

- Only the gateway is public. Databases and workers sit inside the private network, and 4 roles (RBAC) limit what each user sees.
- Teach the concept: **hori ontal scaling**. Monsoon brings 10× the complaints, so add AI workers and the queue absorbs the spike.
- Observability: logs, metrics and alerts, so we know a worker crashed before citizens do.

Point at: the dashed boundary, the external services, the observability strip.

Likely question: Can a field worker see other workers' tasks?
Answer: No. The JWT carries their ID and role, and the API filters by assigned worker.

Design decisions and their trade-offs

Showing what each choice costs is what earns technical depth marks.

RabbitMQ event bus

Slow or failing AI never blocks a citizen.

One more component to run, and harder debugging.

PostgreSQL + PostGIS

Related data, plus fast "nearest" queries.

Migrations, and learning spatial SQL.

Presigned uploads

Large files skip the API.

Orphan files need a nightly clean up.

AI as a separate Python worker

Best ML libraries, and it scales on its own.

Two languages and a shared message format.

Worker GPS in Redis, 60 s expiry

Fast writes, and stale data deletes itself.

No location history unless sampled separately.

Your diagram must include

Include

- ✓ The **name of your architecture style**, and why you chose it over one alternative
- ✓ Every client type, and how each one connects (protocol on the arrow)
- ✓ A single entry point (gateway or API) and what it checks
- ✓ Each backend component, named with its technology
- ✓ Every data store, labelled with what it holds
- ✓ Synchronous and asynchronous paths drawn differently
- ✓ External services (AI APIs, maps, payment, messaging)
- ✓ What is public and what is private, plus a legend

Avoid

- ✗ Arrows without labels
- ✗ A list of technologies with no flow
- ✗ Frontend screens instead of the backend
- ✗ "AI" as one box, with no input, output or place it runs
- ✗ Components you can't explain

- ✕ Text too small to read from the back of the room

How you will be marked (each member individually)

3

Technical depth

You teach a real concept and state a trade-off.

2

Clarity

Clear, within your 2.5 minutes, clean handover.

2

Diagram

Your part of the diagram is complete and readable.

3

Q&A

You answer your question with a reason, not a guess.

Presentations count for **30% of your FYP-1 marks**. Asking good questions when other groups present also earns participation marks.

Bring on 24-Oct

- Your backend architecture diagram as a PDF or PNG, **sent to me the day before** (all groups present from one laptop)
- Your clickable mockups (Figma link) covering all main user flows
- Each member ready with their 2.5 minute part, rehearsed with a timer

If anything is unclear, ask me before 24-Oct.
Abdul Wahab · FYP Advisor